principle-viz 软件包：经济学基本图形

Pin Yue Sung[*†] Ling Tak Douglas Chung[*]

2026-10-23 · v0.12.1 · ·

目录

1 简介 2
1.1 功能范围 2
1.2 阅读指引 2
2 安装 3
2.1 系统需求 3
2.2 安装 uv 3
2.3 安装 principle-viz 3
2.4 安装命令行工具 4
2.5 开发环境设置 4
2.6 验证安装 4
3 快速开始 4
3.1 基本范例 4
3.2 逐步说明 5
3.3 结果 5
3.4 检查输入与求解失败 6
4 线性市场 6
4.1 直线 6
4.2 均衡 7
4.3 比较静态 7
4.4 异常 9
5 离散市场 9
5.1 逐单位表 9
5.2 均衡 9
5.3 图形 10
6 由个人加总市场曲线 11
6.1 加总线性曲线 11
6.2 加总离散表 12
6.3 加总图 12
7 弹性与总收益 14
7.1 价格弹性 14
7.2 总收益 14
8 福利 15
8.1 市场结果 15
8.2 剩余 16
8.3 图形 16
8.4 无谓损失报表 17
9 税收与补贴 17
9.1 情景 17
9.2 求解 18
9.3 图形 19
9.4 补贴 20
10 价格管制 21
11 国际贸易 24
12 市场失灵 26
12.1 外部性 26
12.2 公共资源 27
12.3 公共物品 28
13 劳动与可贷资金 28
13.1 最低工资 28
13.2 可贷资金 29
14 生产可能性曲线 30
14.1 比较优势 31
15 图形 32
15.1 MarketFigure 32
15.2 标签与图层 33
15.3 配色与主题 35
15.4 配置文件 36
15.5 画布 37
16 命令行界面 37
16.1 市场参数 38
16.2 命令 38
更新纪录 40
命令索引 42
参考文献 44

[*]台湾政治大学国际经营与贸易学系，台北，台湾。
[†]通讯作者。电子邮箱：contact@econ-viz.org

第 1 节　简介

“0.10.0”,)[由 `principle-econ` 更名为 principle-viz，加入 EconViz 系列；Python 软件包名称改为 `principle_viz`]

principle-viz 是经济学原理的绘图软件包，涵盖供给与需求、均衡及其移动、税收与补贴、价格管制、福利、国际贸易、市场失灵、要素市场与生产可能性曲线。每个主题都有计算与图形两个部分：计算返回数值，图形依教科书惯例绘制。

输出格式包括 PNG、SVG 与 PDF；命令行工具则以 JSON 输出计算结果。

principle-viz 属于 EconViz 系列软件包，绘图使用与领域无关的场景与绘制函数库 mosaickit。同系列的 utility-viz 是个体经济学绘图软件包，涵盖效用模型、最优消费组合求解，以及无异曲线、预算线、消费者均衡、需求曲线与 Edgeworth 箱形图；本软件包不包含这些功能。

1.1　功能范围

principle-viz 处理线性市场，功能分为市场、政策、应用与工具四个部分。计算与图形可分开使用；同一个结果可以打印、输出为 JSON，也可以交给 `MarketFigure` 绘制。

1.2　阅读指引

本手册依主题分成市场、政策、应用与工具四个部分（参见表 1）：

表 1
章节主题

主题	子主题	说明	章节
市场	线性市场	直线、均衡与曲线移动	第 4 节
	离散市场	逐单位的需求与供给表	第 5 节
	市场曲线加总	个人曲线的水平加总	第 6 节
	弹性与总收益	点弹性、弧弹性与总收益曲线	第 7 节
政策	福利	消费者剩余、生产者剩余与无谓损失	第 8 节
	税收与补贴	税负楔子与补贴成本	第 9 节
	价格管制	价格上限、价格下限与短缺	第 10 节
应用	国际贸易	自由贸易、关税与进口配额	第 11 节
	市场失灵	外部性、公共资源与公共物品	第 12 节
	劳动与可贷资金	最低工资与政府借款	第 13 节
	生产可能性曲线	机会成本、成长与比较优势	第 14 节
工具	图形	标签、图层与配色	第 15 节
	命令行接口	以 JSON 输出计算结果	第 16 节

初次使用时，依序阅读第 3 节、第 4 节与第 15 节。之后每章说明一个主题，先介绍计算，再介绍图形。查找特定命令的选项时，可直接前往第 16 节。

绘制本手册各图的脚本按主题分类，可一次下载：principle-viz-examples.zip。

第 2 节　安装

2.1　系统需求

principle-viz 需要 Python 3.10 以上版本[1]。唯一的运行期依赖是 mosaickit，它通过 matplotlib 绘制图形；计算部分只使用标准函数库。

2.2　安装 uv

本手册的命令以 uv[2] 为准。既有项目仍可使用 pip、pipx 或 Poetry；软件包 API 不受管理工具影响。

依操作系统运行下列安装命令。

macOS、Linux

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Windows

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/
install.ps1 | iex"
```

2.3　安装 principle-viz

创建项目并加入软件包：

```
uv init my-diagrams
cd my-diagrams
uv add principle-viz
uv run python main.py
```

将第 3 节的基本范例存为项目目录下的 `main.py`，再运行上述命令。`uv add` 记录项目依赖，`uv run` 使用该项目的 Python 环境。安装与运行必须使用同一个环境。

若要固定本手册使用的版本，可在加入依赖时指定版本号：

```
uv add "principle-viz==0.12.1"
```

若使用既有的 Python 虚拟环境，可通过 pip 安装：

```
python -m pip install -U principle-viz
```

发行软件包名称是 `principle-viz`，Python 导入名称是 `principle_viz`：

```
import principle_viz
from principle_viz import solve_equilibrium, MarketFigure
```

`import` 陈述式不得使用连字号。若发生 `ModuleNotFoundError`，检查运行程序的解释器是否与安装软件包时使用的环境相同。

0.10.0 版以前，本软件包以 `principle-econ` 名称发布。该发行软件包已停止更新；请改装 `principle-viz`，并将导入的 `principle_econ` 改为 `principle_viz`。

---

[1] Python 官方网站提供各操作系统的安装程序：`https://www.python.org/downloads/`。

[2] uv 是 Astral 开发的 Python 软件包与项目管理工具，速度快且可一并管理 Python 版本；安装方式与完整说明见官方文档：`https://docs.astral.sh/uv/`。

2.4　安装命令行工具

仅使用命令行接口时，可将 principle-viz 安装为独立工具（详见第 16 节）：

```
uv tool install principle-viz
principle-viz equilibrium --demand-intercept 10 --demand-slope -1 \
                          --supply-intercept 2 --supply-slope 1
```

此方式将命令行工具安装在独立环境。需要在 Python 程序中导入软件包时，仍须在该项目运行 `uv add principle-viz`。在项目内以 `uv run principle-viz` 调用工具，可让命令行与 Python 程序使用同一版本。

2.5　开发环境设置

```
git clone https://github.com/EconViz/principle-viz.git
cd principle-viz
uv sync
uv run pytest -q
uv run ruff check src tests examples/scripts
```

测试要求至少 90% 的陈述式覆盖率。范例脚本会将项目图库中的所有图形写入 `examples/output/`：

```
uv run python examples/scripts/run_all.py
```

2.6　验证安装

以下命令确认 Python 能导入求解与绘图接口：

```
uv run python -c "import principle_viz; print('OK')"
```

命令行工具以 `uv run principle-viz --help` 确认，会列出所有命令。在服务器或其他没有图形接口的环境中，请以 `save()` 输出文件。

第 3 节　快速开始

3.1　基本范例

本章以一个市场说明完整流程，后续各章也沿用这个市场：

$$\text{需求：}\quad p = 10 - Q, \qquad \text{供给：}\quad p = 2 + Q.$$

例 1

```python
from principle_viz import MarketFigure, line_from_inverse, solve_equilibrium

demand = line_from_inverse(10.0, -1.0)   # p = 10 - Q
supply = line_from_inverse(2.0, 1.0)     # p = 2 + Q
eq = solve_equilibrium(demand, supply)
print(eq.q_star, eq.p_star)              # 4.0 6.0

fig = MarketFigure(x_max=12, y_max=12, title="Basic Equilibrium")
fig.add_curves(demand, supply, q_max=10)
fig.add_equilibrium(eq)
fig.finalize()
fig.save("basic_equilibrium.png")
```

由 $10 - Q = 2 + Q$ 得 $Q^* = 4$、$p^* = 6$。输出参见图 1。

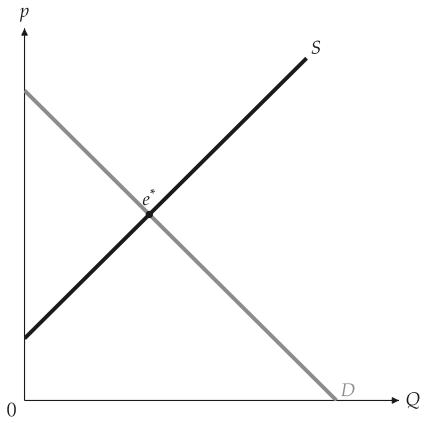


图 1
本章的市场

3.2　逐步说明

每张图依序经过描述市场、求解、绘图、完成并保存四个步骤。

`line_from_inverse`　`line_from_inverse(⟨浮点数⟩, ⟨浮点数⟩)`
描述市场。`line_from_inverse(a, b)` 创建直线 $p = a + bQ$（详见第 4 节）。

`solve_equilibrium`　`solve_equilibrium(⟨需求⟩, ⟨供给⟩)`
求解。`solve_equilibrium()` 等求解函数返回由数值组成的不可变 dataclass，此时尚未绘图。

`MarketFigure`　`MarketFigure(x_max=⟨浮点数⟩, y_max=⟨浮点数⟩, ...)`
绘图。`MarketFigure` 是含价格轴与数量轴的正方形图。坐标范围只控制显示区域，不参与计算；均衡点未出现在图中时，检查 `eq.q_star` 与 `eq.p_star` 是否超出范围，再调整坐标上限。

`MarketFigure.add_*`　`add_*` 方法接收曲线与结果，加入对应的图层；每个方法都返回图形本身，可串接调用。

`MarketFigure.finalize`
`MarketFigure.save`　完成并保存。`finalize()` 隐藏会把阴影面积切成两半的辅助线；`save()` 依扩展名写出 PNG、SVG 或 PDF，并创建不存在的目录（详见第 15 节）。

计算与绘图互不依赖：结果可以不经绘图直接打印、比较或导出，同一个结果也可以画在多张图上。

3.3　结果

结果是不可变的 dataclass，字段为浮点数、字符串与 tuple。`solve_equilibrium()` 返回 `EquilibriumResult`：

`q_star`　均衡数量 Q^*。
float

`p_star`　均衡价格 p^*。
float

`is_valid_market`　均衡数量为负时为 `False`；价格为负只会加入一则说明。
bool

notes　说明解的特殊情况。

tuple[*str*, ...]

3.4　检查输入与求解失败

PrincipleVizError　软件包抛出的所有异常都继承自 `principle_viz.exceptions` 中的 `PrincipleVizError`，捕获它即可一并处理（各异常详见第 4.4 节）。

两条直线平行时没有均衡，`solve_equilibrium()` 抛出 `ParallelLinesError`，不会返回无效的结果：

```
from principle_viz import line_from_inverse, solve_equilibrium
from principle_viz.exceptions import PrincipleVizError

demand = line_from_inverse(10.0, -1.0)
supply = line_from_inverse(8.0, -1.0)   # parallel to demand

try:
    eq = solve_equilibrium(demand, supply)
except PrincipleVizError as error:
    print(type(error).__name__)
else:
    print(eq.q_star, eq.p_star)

# ParallelLinesError
```

均衡数量为负时不抛出异常，而是将 `is_valid_market` 设为 `False`。使用结果前，先检查 `is_valid_market` 与 `notes`。

第 4 节　线性市场

4.1　直线

需求或供给曲线以反函数形式表示为直线：

$$p = a + bQ$$

其中 a 为价格截距，b 为斜率（需求 $b < 0$，供给 $b > 0$）。价格一律在纵轴、数量在横轴，沿用 Marshall (1890) 的画法。离散的逐单位表（详见第 5 节）与由个人曲线加总而成的分段线性市场曲线（详见第 6 节）不是单一直线。

Line　`Line.from_inverse(⟨浮点数⟩, ⟨浮点数⟩)`

`Line.from_standard(⟨浮点数⟩, ⟨浮点数⟩, ⟨浮点数⟩)`

价格—数量平面上的直线，内部保存为 $Ap + BQ + C = 0$，因此也能表示水平线与垂直线。`from_inverse(a, b)` 创建 $p = a + bQ$；`from_standard(A, B, C)` 创建 $Ap + BQ + C = 0$。软件包根目录另以 `line_from_inverse()` 与 `line_from_standard()` 提供这两个构造函数。

p_at(q)　数量 q 时的价格。

q_at(p)　价格 p 时的数量。

p_intercept()　$Q = 0$ 时的价格（需求曲线的阻绝价格）。

q_intercept()　$p = 0$ 时的数量。

`slope()` $\mathrm{d}p/\mathrm{d}Q$。

`to_inverse()` $p = a + bQ$ 的 `(a, b)`。

`shifted()` 将截距与斜率分别加上 `delta_intercept`（Δa）与 `delta_slope`（Δb）后的新直线。

```
from principle_viz import line_from_inverse, line_from_standard

demand = line_from_inverse(10.0, -1.0)
print(demand.q_at(4), demand.p_at(3))          # 6.0 7.0
print(demand.p_intercept(), demand.q_intercept())  # 10.0 10.0
print(line_from_standard(1, 1, -10).to_inverse())  # (10.0, -1.0)
```

$(A, B, C) = (1, 1, -10)$ 与 $p = 10 - Q$ 等价。

水平线没有 $Q(p)$，垂直线没有 $p(Q)$；调用对应的方法会抛出 `NonInvertibleLineError`。

4.2　均衡

`solve_equilibrium` `solve_equilibrium(⟨需求⟩, ⟨供给⟩)`

两条直线的交点，返回 `EquilibriumResult`（详见第 3 节）。两线平行时抛出 `ParallelLinesError`，重合时抛出 `CoincidentLinesError`。

```
from principle_viz import solve_equilibrium

eq = solve_equilibrium(demand, line_from_inverse(2.0, 1.0))
print(eq)
# EquilibriumResult(q_star=4.0, p_star=6.0,
#                   is_valid_market=True, notes=())
```

由 $10 - Q = 2 + Q$ 得均衡数量 4、均衡价格 6，`is_valid_market` 为 `True`，`notes` 为空。

4.3　比较静态

`ShiftSpec` `ShiftSpec(delta_intercept=0.0, delta_slope=0.0)`

`ShiftScenario` `ShiftScenario(demand_shift=None, supply_shift=None)`

以反函数形式描述一条曲线的移动：`delta_intercept` 为正时向上平移、为负时向下，`delta_slope` 旋转曲线。情景可同时移动需求、供给或两者。两者都位于 `principle_viz.core.shifts`。

需求增加使需求截距上升；供给增加则使供给截距下降，因为卖方在每个数量下都愿意接受较低的价格。

`comparative_statics` `comparative_statics(⟨需求⟩, ⟨供给⟩, ⟨情景⟩)`

求解移动前后的市场，结果包含下列字段：

`baseline_equilibrium` 移动前的 `EquilibriumResult`。

`shifted_equilibrium` 移动后的 `EquilibriumResult`。

`shifted_market` 移动前后的直线（`baseline_demand`、`shifted_demand` 等）。

`delta_q` 数量的变动。

`delta_p` 价格的变动。

`direction_q` 数量的变动方向：`"left"` 或 `"right"`（未变动时为 `"none"`）。

`direction_p` 价格的变动方向：`"up"` 或 `"down"`（未变动时为 `"none"`）。

```
from principle_viz import comparative_statics
from principle_viz.core.shifts import ShiftScenario, ShiftSpec
```

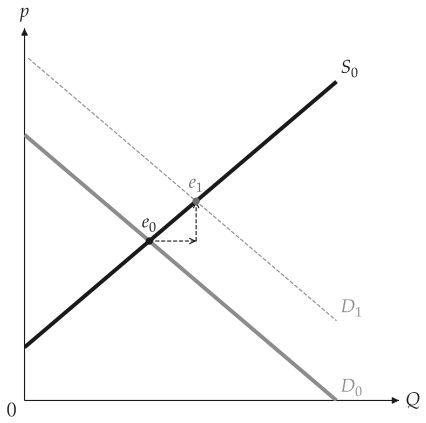


图 2
需求增加

```
up = ShiftScenario(demand_shift=ShiftSpec(delta_intercept=3.0))
result = comparative_statics(demand, supply, up)
new = result.shifted_equilibrium
print(new.q_star, new.p_star)                    # 5.5 7.5
print(result.direction_q, result.direction_p)  # right up
```

MarketFigure.add_comparative_statics

```
add_comparative_statics(⟨结果⟩, q_max, *, demand_label="$D_1$",
supply_label="$S_1$")
```

画出移动的曲线、两个均衡点，以及由旧均衡指向新均衡的虚线箭头。在 `add_curves()` 中将原曲线命名为 D_0 与 S_0。需求增加的结果如图 2，供给减少的结果如图 3。

```
fig = MarketFigure(x_max=12, y_max=14, title="Increase in Demand")
fig.add_curves(
    demand, supply, q_max=10,
    demand_label="$D_0$", supply_label="$S_0$",
)
fig.add_comparative_statics(result, q_max=10)
fig.finalize()
```

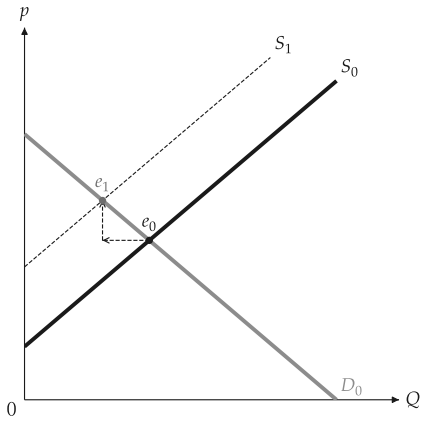


图 3
供给减少

4.4　异常

软件包抛出的所有异常都继承自 `principle_viz.exceptions` 中的 `PrincipleVizError`，捕获它即可一并处理。0.10.0 版以前的名称 `PrincipleEconError` 是同一个类别。各异常的抛出时机如表 2。

表 2　异常类别

异常	抛出时机
`LineError`	无法创建或转换直线
`NonInvertibleLineError`	对水平线求 $Q(p)$，或对垂直线求 $p(Q)$
`ParallelLinesError`	需求与供给没有交点
`CoincidentLinesError`	需求与供给是同一条直线
`PolicyError`	税收、补贴、价格管制或贸易情景无效
`DiscreteMarketError`	离散逐单位表无效（详见第 5 节）
`AggregationError`、`PiecewiseLinearError`	个人曲线无法加总，或价格超出分段曲线范围（详见第 6 节）
`PPFError`	生产可能性曲线无效（详见第 14 节）

第 5 节　离散市场

离散市场逐一列出每个单位：买方对多一单位的愿付价格，以及卖方多生产一单位的成本。需求与供给都是阶梯函数；均衡是整数个单位，并由一段价格区间支持。

5.1　逐单位表

`DiscreteDemand` `DiscreteDemand(⟨数值⟩)`

`DiscreteSupply` `DiscreteSupply(⟨数值⟩)`

需求表存放边际愿付价格，由第一单位到最后一单位弱递减；供给表存放边际成本，弱递增。顺序不符时抛出 `DiscreteMarketError`。

`quantity_at(price)` 价格为 `price` 时的成交单位数；在该价格下无差异的买方或卖方会交易。

`unit_count` 表中的单位数。

`combine(*schedules)` 将多个个人表合并为市场表（详见第 6 节）。

5.2　均衡

`solve_discrete_equilibrium` `solve_discrete_equilibrium(⟨需求⟩, ⟨供给⟩, *, price_rule="midpoint")`

交易所有价值不低于成本的单位，并找出恰好使该数量成交的价格区间。结果包含下列字段：

`q_star` 成交单位数。

int

`price_low` 支持均衡的价格区间下限。

float

`price_high` 支持均衡的价格区间上限。

float

`price` 依 `price_rule` 从区间中选出的价格。

float

`price_rule` `MIDPOINT`（默认）、`LOWER` 或 `UPPER`；也可传入字符串 `"midpoint"`、`"lower"`、`"upper"`。

EquilibriumPriceRule

`traded_values` 成交单位的价值。

tuple

`traded_costs` 成交单位的成本。

tuple

`gains_from_trade` 每个成交单位的价值减成本。

tuple

`is_unique_price` 区间是否只有单一价格。

bool

`solve_discrete_market(demand_values, supply_values)` 直接由 tuple 创建两张表并求解。需求表的值由高到低、供给表的值由低到高，各单位的价值与成本才能依序配对。

```
from principle_viz import (
    DiscreteDemand, DiscreteSupply, solve_discrete_equilibrium,
)

demand = DiscreteDemand((11, 9, 7, 5, 3))
supply = DiscreteSupply((1, 3, 5, 8, 10))
eq = solve_discrete_equilibrium(demand, supply)
print(eq.q_star, eq.price_low, eq.price_high, eq.price)
# 3 5.0 7.0 6.0
print(eq.gains_from_trade)
# (10.0, 6.0, 2.0)
```

成交 3 单位，价格区间为 5 到 7，中点规则给出 6。交易利得合计 $10 + 6 + 2 = 18$。

`compute_discrete_surplus` `compute_discrete_surplus(⟨结果⟩)`

所选价格下的消费者剩余与生产者剩余，包含总额与逐单位数值（`consumer_surplus_by_unit`、`producer_surplus_by_unit`）。上例价格为 6 时，两者皆为 $5 + 3 + 1 = 9$。

5.3　图形

`MarketFigure.add_discrete_curves` `add_discrete_curves(demand=None, supply=None, *, demand_label="$D$", supply_label="$S$")`

`MarketFigure.add_discrete_equilibrium` `add_discrete_equilibrium(⟨结果⟩)`

每个单位画成区间 $[q, q+1)$ 的一阶：起点为实心点（包含），终点为空心点（不包含），并以虚线连到下一阶。可只传入其中一张表。均衡在坐标轴上标出 Q^* 与价格区间。

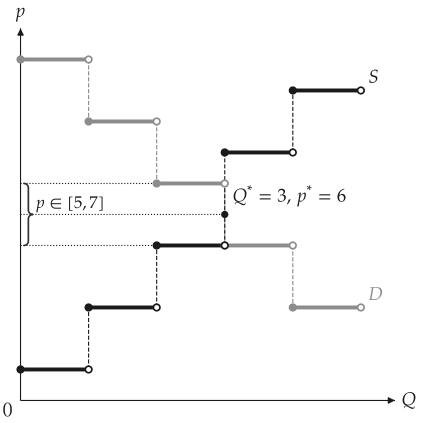


图 4
离散需求与供给

```
fig = MarketFigure(
    x_max=5.5, y_max=12, title="Discrete Demand and Supply",
)
fig.add_discrete_curves(demand, supply)
fig.add_discrete_equilibrium(eq)
fig.finalize()
```

上例的图形参见图 4。

第 6 节　由个人加总市场曲线

市场曲线是个人曲线的水平加总：在每个价格下，市场数量等于所有买方（或卖方）在该价格下选择的数量之和。买方在阻绝价格以上不购买，卖方在最低价格以下不销售。线性个人曲线加总后的市场曲线是分段线性的，每多一人进入市场出现一个折点。

6.1　加总线性曲线

`market_demand`　`market_demand(⟨个人曲线⟩)`

`market_supply`　`market_supply(⟨个人曲线⟩, *, p_max)`
加总负斜率（需求）或正斜率（供给）的个人直线。供给加总由最低的最低价格延伸到 `p_max`。斜率方向不符时抛出 `AggregationError`。

`PiecewiseLinear`　`PiecewiseLinear(⟨points⟩)`
通过 `points` (Q, p) 的需求或供给曲线，点依数量排序。若第一点的 $Q = 0$，超出该点的价格下数量为零。

`q_at(p)`　价格对应的数量；超出曲线范围时抛出 `PiecewiseLinearError`。
`p_at(q)`　数量对应的价格；超出曲线范围时抛出 `PiecewiseLinearError`。
`points`　所有顶点。
`kinks`　斜率改变的内部顶点。
`domain`　涵盖的数量范围。
`price_range`　涵盖的价格范围。
`breakpoints()`　顶点价格，由小到大排列。
`integrate_q()`　`p_low` 与 `p_high` 两个价格之间 $Q(p)$ 下方的面积。

```python
from principle_viz import (
    line_from_inverse, market_demand, market_supply,
)

a = line_from_inverse(10, -2)    # p = 10 - 2Q
b = line_from_inverse(6, -0.5)   # p = 6 - 0.5Q
demand = market_demand((a, b))
print(demand.points)   # ((0.0, 10.0), (2.0, 6.0), (17.0, 0.0))
print(demand.q_at(4))  # 7.0 = Q_A + Q_B = 3 + 4
```

买方 B 在价格 6 进入市场，折点为 $(2, 6)$。价格为 4 时，$Q_A = 3$、$Q_B = 4$，市场需求量为 7。

solve_piecewise_equilibrium

`solve_piecewise_equilibrium(⟨需求⟩, ⟨供给⟩)`

piecewise_surplus

`piecewise_surplus(⟨需求⟩, ⟨供给⟩, ⟨结果⟩)`

逐段精确求解的市场均衡，以及沿价格积分得到的消费者剩余与生产者剩余。

```python
from principle_viz import (
    piecewise_surplus, solve_piecewise_equilibrium,
)

c = line_from_inverse(2, 1)
d = line_from_inverse(5, 0.5)
supply = market_supply((c, d), p_max=10)
eq = solve_piecewise_equilibrium(demand, supply)
print(round(eq.q_star, 3), round(eq.p_star, 3))  # 3.818 5.273
cs, ps = piecewise_surplus(demand, supply, eq)
```

6.2　加总离散表

`DiscreteDemand.combine(*schedules)` 将所有买方的保留价格合并为一张市场表，由高到低排列；`DiscreteSupply.combine()` 合并单位成本，由低到高排列。相同的数值保留为不同单位，合并结果可直接交给 `solve_discrete_equilibrium()`（详见第 5 节）。输出参见图 6。

```python
from principle_viz import DiscreteDemand

first = DiscreteDemand((10, 7, 4))
second = DiscreteDemand((8, 5, 2))
market = DiscreteDemand.combine(first, second)
print(market.values)
# (10.0, 8.0, 7.0, 5.0, 4.0, 2.0)
```

6.3　加总图

demand_aggregation_figure

`demand_aggregation_figure(⟨个人曲线⟩, *, price, price_label="$p_1$", link_price=False, ...)`

supply_aggregation_figure

`supply_aggregation_figure(⟨个人曲线⟩, *, price, p_max, ...)`

discrete_demand_aggregation_figure

`discrete_demand_aggregation_figure(⟨个人曲线⟩, *, price, ...)`

discrete_supply_aggregation_figure

`discrete_supply_aggregation_figure(⟨个人曲线⟩, *, price, ...)`

discrete_supply_aggregation_figure

新增：*v0.10.0*

每位个人一个面板，最后是市场面板，左右并排并共用价格轴。`individuals` 将名称对应到曲线或逐单位表；曲线命名为 D_A、D_B……与 D（或 S_A……与 S）。在 `price` 处以虚线标出 Q_A、Q_B 与 $Q_A + Q_B = Q$。`supply_aggregation_figure()` 另需 `p_max`。

`link_price`
让价格线横跨所有面板及面板间的空隙，只在第一个面板标出价格。

bool · 默认 `False`

`theme`
与 `MarketFigure` 相同（详见第 15.3 节）。

`palette`
与 `MarketFigure` 相同（详见第 15.3 节）。

`labels`
依图层 id 覆盖标签（详见第 15.2 节）。

`visibility`
依图层 id 覆盖图层的显示状态（详见第 15.2 节）。

返回的 `AggregationFigure` 与 `MarketFigure` 一样提供 `save()`、`hide()`、`show()`、`configure_label()`、`layer_ids` 与 `label_ids`。

```python
from principle_viz import demand_aggregation_figure

fig = demand_aggregation_figure(
    {"A": a, "B": b}, price=4, link_price=True,
)
fig.save("market_demand.png")
```

`demand_aggregation_figure()` 的输出参见图 5。

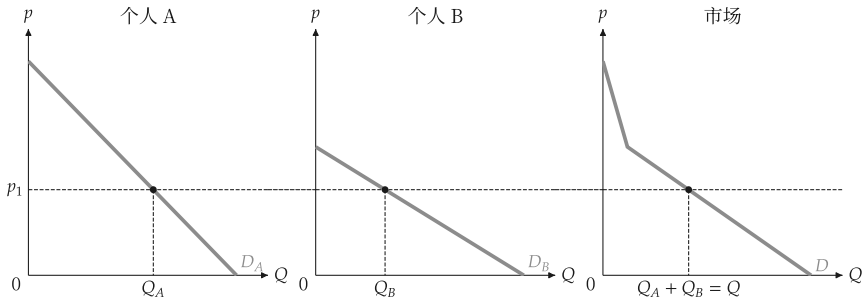


图 5
水平加总的市场需求

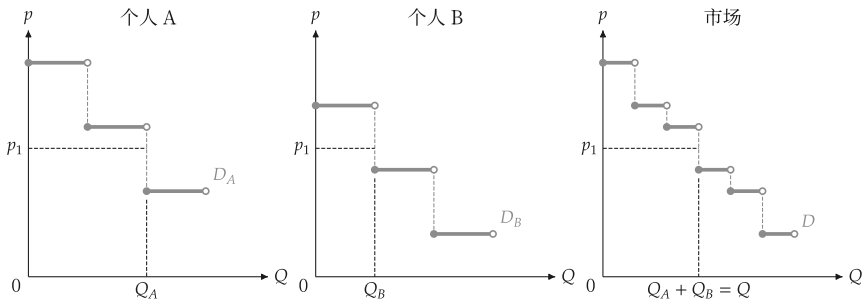


图 6
合并两张离散需求表

第 7 节　弹性与总收益

7.1　价格弹性

compute_point_elasticity(⟨需求⟩, ⟨*quantity*⟩)

compute_arc_elasticity(q0, p0, q1, p1)

直线在某数量下的点价格弹性

$$\varepsilon = \frac{\mathrm{d}Q}{\mathrm{d}p}\cdot\frac{p}{Q},$$

以及两点之间的弧弹性（中点法）

$$\varepsilon = \frac{\Delta Q / \overline{Q}}{\Delta p / \overline{p}},$$

其中 $\overline{Q}$ 与 $\overline{p}$ 为两点的平均值。负斜率需求的弹性皆为负值。

```python
from principle_viz import (
    compute_arc_elasticity, compute_point_elasticity,
line_from_inverse,
)

demand = line_from_inverse(10.0, -1.0)
print(compute_point_elasticity(demand, 4))   # -1.5
print(compute_point_elasticity(demand, 5))   # -1.0 (unit elastic)
print(compute_arc_elasticity(4, 6, 6, 4))    # -1.0
```

$Q = 4$ 处弹性为 -1.5，绝对值大于 1，需求有弹性；$Q = 5$ 为单位弹性，也是 $p = 10 - Q$ 的中点。弧弹性以两点的平均值计算，因此由 $(4, 6)$ 到 $(6, 4)$ 与反方向的结果相同。

`principle_viz.core.elasticity` 中的 `classify_elasticity(value)` 依绝对值分类：大于 1 为 `"elastic"`，等于 1 为 `"unit_elastic"`，小于 1 为 `"inelastic"`。

7.2　总收益

elasticity_revenue_schedule(⟨需求⟩, *, samples=101)

在线性需求上由 $Q = 0$ 采样到阻绝数量，返回每一点的价格、总收益 pQ、弹性与其分类。总收益在单位弹性处（直线的中点）最大。

points　RevenuePoint 串行，含 quantity、price、total_revenue、elasticity 与 classification。

unit_elastic_quantity　直线中点的数量。

unit_elastic_price　直线中点的价格。

maximum_revenue　该点的总收益。

choke_quantity　数量轴截距。

choke_price　价格轴截距。

$p = 12 - Q$ 的总收益最大值为 36，位于 $Q = 6$、$p = 6$。

elasticity_revenue_canvases

`elasticity_revenue_canvases(⟨需求⟩, ⟨结果⟩, *, theme=None, labels=None, visibility=None)`

返回两张 mosaickit 画布，位于 `principle_viz.visuals.revenue`：一张是需求曲线，沿线标出 “Elastic”、“Unit elastic” 与 “Inelastic”，并标记单位弹性点；另一张是总收益对数量的曲线，标记最大值。两张画布各有标题。以 `mosaickit.CanvasGrid` 左右并排（参见图 7）。

```
from mosaickit import CanvasGrid
from principle_viz import PlotTheme, elasticity_revenue_schedule
from principle_viz.visuals.revenue import (
    elasticity_revenue_canvases,
)

demand = line_from_inverse(12.0, -1.0)
schedule = elasticity_revenue_schedule(demand)
canvases = elasticity_revenue_canvases(
    demand, schedule, theme=PlotTheme(),
)
CanvasGrid(canvases, rows=1).save("elasticity_total_revenue.png")
```

第 8 节　福利

福利分析比较某个市场结果与其他可能结果：消费者剩余、生产者剩余、政府收入，以及无谓损失。

8.1　市场结果

MarketOutcome

`MarketOutcome(quantity, consumer_price, producer_price, label="")`

一个数量，加上买方支付与卖方收到的价格。征税或补贴时两个价格不同，价差乘以数量就是政府收入（补贴为负）。此类别位于 `principle_viz.welfare.surplus`，同模块也提供由各种结果创建市场结果的函数：

outcome_from_equilibrium(eq)

自由市场：两个价格都是 p^*。

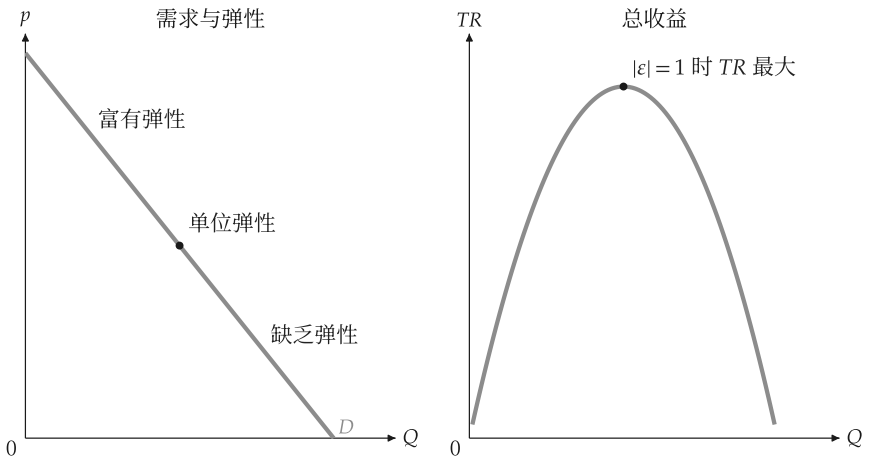


图 7
弹性与总收益

`outcome_from_tax(tax_eq)` 征税后（详见第 9 节）。

`outcome_from_subsidy(subsidy_eq)` 补贴后。

`outcome_from_control(control)` 价格上限或下限之下（详见第 10 节）。

8.2　剩余

`compute_surplus` `compute_surplus(⟨需求⟩, ⟨供给⟩, ⟨市场结果⟩, baseline_outcome=None)`
`outcome` 下的福利分解。传入 `baseline_outcome` 时，以其为基准衡量无谓损失。返回的 `SurplusResult` 包含下列字段：

`consumer_surplus` 需求曲线与消费者价格之间的面积。

float
`producer_surplus` 生产者价格与供给曲线之间的面积。

float
`tax_revenue` 政府收入（补贴为负）。

float
`total_surplus` 三者之和。

float
`deadweight_loss` 相对于基准损失的剩余。

float
`polygons` 各面积的顶点，供绘图使用。

SurplusPolygons

```python
from principle_viz import compute_surplus, solve_equilibrium
from principle_viz.welfare.surplus import outcome_from_equilibrium

eq = solve_equilibrium(demand, supply)
outcome = outcome_from_equilibrium(eq)
surplus = compute_surplus(demand, supply, outcome)
print(surplus.consumer_surplus, surplus.producer_surplus)  # 8.0 8.0
```

$(10-6)\times 4/2 = (6-2)\times 4/2 = 8$；自由市场的无谓损失为 0。

`compare_surplus` `compare_surplus(⟨需求⟩, ⟨供给⟩, baseline_outcome, policy_outcome)`
两个福利分解及其差异：`baseline`、`policy`、`delta_consumer_surplus`、`delta_producer_surplus`、`delta_tax_revenue`、`delta_total_surplus` 与 `deadweight_loss`。第 9 节以税收为例使用此函数。

8.3　图形

`MarketFigure.add_welfare` `add_welfare(⟨结果⟩, *, labels=True, regions=None)`

为消费者剩余、生产者剩余、税收与无谓损失填色并命名：名称放得下时写在区块内，只放得下缩写时写 CS、PS、Tax、DWL，否则以引线标示，且不遮住任何线、点或其他文字。

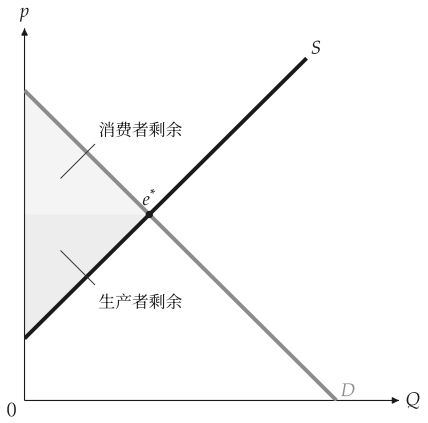


图 8
均衡下的剩余

`labels`
bool · 默认 `True`

设为 `False` 时只填色、不命名。

`regions`
iterable | *None* · 默认 `None`

只画 `"cs"`、`"ps"`、`"tax_revenue"` 与 `"dwl"` 中的部分区块。

```
fig = MarketFigure(x_max=12, y_max=12)
fig.add_curves(demand, supply, q_max=10)
fig.add_welfare(surplus)
fig.add_equilibrium(eq)
fig.finalize()
```

输出参见图 8。

`MarketFigure.add_welfare_transition`

`add_welfare_transition(*, baseline_outcome, policy_outcome, surplus)`

同样的区块，再加上基准与政策下数量及价格的辅助线，用于比较前后差异。

8.4　无谓损失报表

`build_dwl_report`

`build_dwl_report(⟨rows⟩)`

将（名称，基准，政策）三元组（皆为 `SurplusResult`）转为报表列（`DWLScenarioRow`），包含数量变动、各项剩余变动与无谓损失。`principle_viz.welfare.report` 中的 `save_dwl_report_csv()` 与 `save_dwl_report_json()` 可将报表写成文件。

第 9 节　税收与补贴

税收在买方支付的价格与卖方收到的价格之间形成楔子。无论向买方或卖方征收，楔子、数量与两个价格都相同 (Mankiw, 2021)。

9.1　情景

`TaxScenario`

`TaxScenario(tax_type, amount, tax_on=TaxOn.PRODUCER, anchor_mode=AnchorMode.NONE)`

一项税收，位于 `principle_viz.policy.tax`。

`tax_type`
TaxType
`amount`
float
`tax_on`
TaxOn · 默认 `PRODUCER`
`anchor_mode`
AnchorMode · 默认 `NONE`

`FIXED_TAX` 或 `PER_UNIT_TAX`：每单位征收固定金额，曲线平移 `amount`；`AD_VALOREM_TAX`：依价格征收比率，曲线旋转。两种从量形式的均衡相同。

每单位税额，或从价税的税率（`0.35` 即 35%）；税率必须大于 -1。

法定纳税人：`PRODUCER` 使供给上移，`CONSUMER` 使需求下移。

绘制从价税时，`NONE` 以截距为轴旋转曲线，`BASELINE_EQUILIBRIUM` 以未征税的均衡点为轴。

9.2　求解

`TaxType`
TaxType

`TaxType.FIXED_TAX, PER_UNIT_TAX, AD_VALOREM_TAX`
税的定义，来自 `principle_viz.policy.tax`。

`FIXED_TAX`
TaxType

固定金额，使曲线平移 `amount`。

`PER_UNIT_TAX`
TaxType

每单位的税额；均衡与 `FIXED_TAX` 相同。

`AD_VALOREM_TAX`
TaxType

按价格计的税率，使曲线旋转。

`TaxOn`
TaxOn

`TaxOn.CONSUMER, PRODUCER`
税的法律归宿，来自 `principle_viz.policy.tax`。

`CONSUMER`
TaxOn

向买方征收：需求向下平移。

`PRODUCER`
TaxOn

向卖方征收：供给向上平移。

`solve_tax_equilibrium`

`solve_tax_equilibrium(⟨需求⟩, ⟨供给⟩, ⟨情景⟩)`
征税后的市场，返回 `TaxEquilibriumResult`：`q_star`、`consumer_price`、`producer_price`、`tax_wedge`（两价之差）与 `tax_revenue`（楔子乘以数量）。

```
from principle_viz import solve_tax_equilibrium
from principle_viz.policy.tax import TaxOn, TaxScenario, TaxType

tax = TaxScenario(TaxType.PER_UNIT_TAX, 3.0, TaxOn.PRODUCER)
print(solve_tax_equilibrium(demand, supply, tax))
# TaxEquilibriumResult(q_star=2.5, consumer_price=7.5,
#                      producer_price=4.5, tax_wedge=3.0,
#                      tax_revenue=7.5)
```

`TaxOn.CONSUMER` 的结果相同。税率为 r 的从价税使消费者价格等于生产者价格的 $(1+r)$ 倍。

`compare_tax_scenario`

`compare_tax_scenario(⟨需求⟩, ⟨供给⟩, ⟨情景⟩)`
并列征税前后的市场：`baseline_equilibrium`、`post_tax`、变动量 `delta_q`、`delta_p_consumer`、`delta_p_producer`，以及各自的方向（`"left"`／`"right"`、`"up"`／`"down"`）。

`build_tax_visual_guide`

`build_tax_visual_guide(⟨需求⟩, ⟨供给⟩, ⟨情景⟩)`

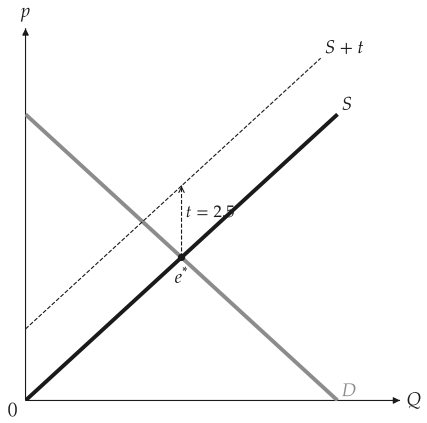


图 9
向卖方征从量税

征税曲线的几何信息：`base_curve`、`taxed_curve`、`curve_role`（`"supply"` 或 `"demand"`）与 `transform_kind`（`"shift"` 或 `"rotation"`）。`add_tax_transform()` 依此绘图。

9.3　图形

`MarketFigure.add_tax_transform`　`add_tax_transform(⟨需求⟩, ⟨供给⟩, ⟨情景⟩, q_max)`

画出征税后的曲线并命名为 $S+t$ 或 $D-t$，以虚线箭头表示平移或旋转，并标出未征税的均衡（参见图 9、图 10）。

`MarketFigure.add_tax_comparison`　`add_tax_comparison(⟨结果⟩, *, brace_side="outside", notes=False)`

标出 `compare_tax_scenario()` 结果的楔子：在价格轴上标 p_d（买方）、p_0（征税前）与 p_s（卖方），并以 “Tax” 括号涵盖 p_s 到 p_d。

`brace_side`
str · 默认 `"outside"`
括号在坐标轴外侧（`"outside"`）或绘图区内（`"inside"`）。

`notes`
bool · 默认 `False`
在坐标轴旁说明每个标记。

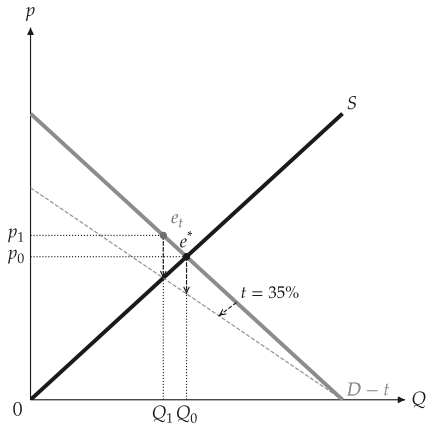


图 10
向买方征从价税

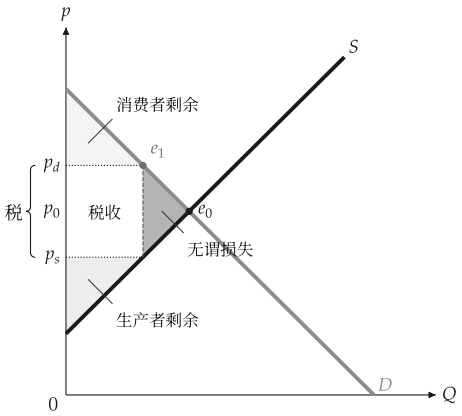


图 11
征税下的剩余与损失

搭配第 8 节的福利区块（参见图 11）：

```
from principle_viz import compare_tax_scenario
from principle_viz.welfare.surplus import (
    compare_surplus, outcome_from_equilibrium, outcome_from_tax,
)

eq = solve_equilibrium(demand, supply)
baseline = outcome_from_equilibrium(eq)
taxed = outcome_from_tax(solve_tax_equilibrium(demand, supply, tax))
delta = compare_surplus(demand, supply, baseline, taxed)
print(delta.policy.tax_revenue, delta.deadweight_loss)  # 7.5 2.25

fig = MarketFigure(x_max=12, y_max=12, title="Welfare Under a Tax")
fig.add_curves(demand, supply, q_max=10)
fig.add_welfare(delta.policy)
fig.add_tax_comparison(compare_tax_scenario(demand, supply, tax))
fig.finalize()
```

9.4　补贴

`SubsidyScenario` — `SubsidyScenario(amount, subsidy_to=SubsidyTo.PRODUCER)`
每单位 `amount` 的补贴（不得为负），发给卖方（`SubsidyTo.PRODUCER`）或买方（`SubsidyTo.CONSUMER`），位于 `principle_viz.policy.subsidy`。

`SubsidyTo` — `SubsidyTo.PRODUCER, CONSUMER`
补贴的受领者，来自 `principle_viz.policy.subsidy`。

`PRODUCER` — 补贴给卖方。
SubsidyTo

`CONSUMER` — 补贴给买方。
SubsidyTo

`solve_subsidy_equilibrium` — `solve_subsidy_equilibrium(⟨需求⟩, ⟨供给⟩, ⟨情景⟩)`

`compare_subsidy_scenario` — `compare_subsidy_scenario(⟨需求⟩, ⟨供给⟩, ⟨情景⟩)`

补贴后的市场（`q_star`、`consumer_price`、`producer_price`、`subsidy_wedge`、`government_expenditure`），以及与自由市场的比较（`baseline_equilibrium`、`post_subsidy`、`delta_q`、`delta_p_consumer`、`delta_p_producer`）。在第 3 节的市场中补贴 2：数量 5，买方支付 5，卖方收到 7，政府支出 10。

`MarketFigure.add_subsidy_comparison`

```
add_subsidy_comparison(〈结果〉, *, brace_side="outside", notes=False)
```

以标示税收楔子的方式标出补贴楔子，加上 “Subsidy” 括号，并标示补贴成本。图 12 以 `regions=("dwl",)` 只为无谓损失填色，并以 `visibility` 隐藏部分标签（详见第 15.2 节）。

```
from principle_viz import (
    SubsidyScenario, SubsidyTo, compare_subsidy_scenario,
)

demand = line_from_inverse(12.0, -1.0)
subsidy = SubsidyScenario(3.0, SubsidyTo.PRODUCER)
comparison = compare_subsidy_scenario(demand, supply, subsidy)
fig = MarketFigure(
    x_max=12, y_max=13, title="Per-Unit Subsidy",
    visibility={
        "market.subsidy.expenditure.label": False,
        "market.subsidy.wedge.label": False,
        "market.subsidy.wedge.mark.p_0": False,
        "market.subsidy.wedge.brace": False,
    },
)
```

第 10 节　价格管制

低于均衡价格的价格上限，或高于均衡价格的价格下限，会产生约束：市场无法结清，成交量等于管制价格下需求量与供给量中较小者。

`PriceControlScenario`

```
PriceControlScenario(control_type, control_price)
```

`PriceControlType`

```
PriceControlType.CEILING | PriceControlType.FLOOR
```

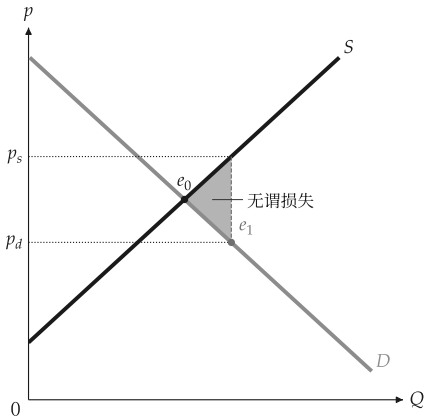


图 12
从量补贴与无谓损失

位于 `control_price` 的上限（`CEILING`）或下限（`FLOOR`），位于 `principle_viz.core.controls`。

`CEILING` 价格上限。

PriceControlType

`FLOOR` 价格下限。

PriceControlType

`evaluate_price_control` `evaluate_price_control(⟨需求⟩, ⟨供给⟩, ⟨情景⟩)`
管制下的市场，返回 `PriceControlResult`：

`is_binding` 管制是否改变市场结果。

bool

`traded_quantity` 实际成交量。

float

`consumer_price` 买方支付的价格：有约束时为管制价格，否则为均衡价格。

float

`producer_price` 卖方收到的价格：有约束时为管制价格，否则为均衡价格。

float

`shortage` 上限下的超额需求（其余情况为零）。

float

`surplus` 下限下的超额供给（其余情况为零）。

float

`baseline_equilibrium` 未管制的市场。

EquilibriumResult

```
from principle_viz import evaluate_price_control
from principle_viz.core.controls import (
    PriceControlScenario, PriceControlType,
)

scenario = PriceControlScenario(PriceControlType.CEILING, 4.0)
ceiling = evaluate_price_control(demand, supply, scenario)
print(ceiling.is_binding, ceiling.traded_quantity, ceiling.shortage)
# True 2.0 4.0
```

上限高于均衡价格（例如 7）时，`is_binding` 为 `False`。

`MarketFigure.add_price_control` `add_price_control(⟨结果⟩, *, gap_brace="line")`

画出管制线并命名为 “Price ceiling” 或 “Price floor”，在价格轴上标出 p_c。有约束时另标出 Q_d 与 Q_s，并以括号标示差额：上限下方为 “Shortage”，下限上方为 “Surplus”。`gap_brace="axis"` 改在数量轴下方标示（参见图 13、图 14）。

`outcome_from_control()` 将结果转为市场结果，可交给第 8 节的福利函数（参见图 15）：

```
from principle_viz.welfare.surplus import (
    compare_surplus, outcome_from_control, outcome_from_equilibrium,
)

eq = solve_equilibrium(demand, supply)
baseline = outcome_from_equilibrium(eq)
controlled = outcome_from_control(ceiling)
```

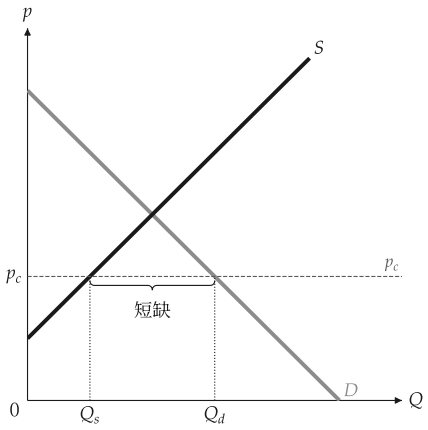


图 13
有约束的价格上限

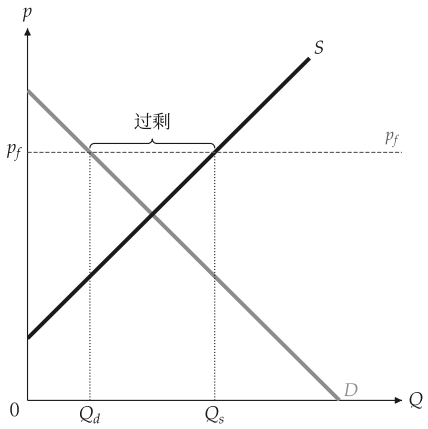


图 14
有约束的价格下限

```
delta = compare_surplus(demand, supply, baseline, controlled)
fig.add_price_control(ceiling)
fig.add_welfare(delta.policy)
```

价格上限下，无谓损失为未成交的交易利益；部分剩余由卖方转移给买方。

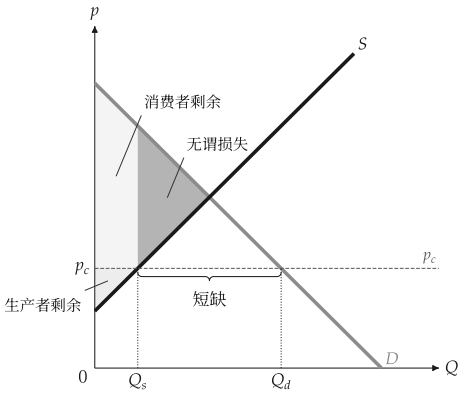


图 15
上限 3.5 时的福利

第 11 节　国际贸易

小型开放经济体接受既定的世界价格 p_w。世界价格低于自给自足价格时，进口国内需求与供给的差额；高于自给自足价格时则出口。关税或进口配额会提高国内价格并减少进口。

`TradeScenario` `TradeScenario(world_price, tariff=0.0, import_quota=None, quota_rent_recipient=QuotaRentRecipient.DOMESTIC)`
世界价格与至多一项政策：每单位 `tariff` 或 `import_quota`，两者不可同时设置（`PolicyError`）。

`quota_rent_recipient` *QuotaRentRecipient* 配额租的归属：国内进口商（`DOMESTIC`）或政府（`GOVERNMENT`），两者都计入国民剩余，政府获得的部分列为收入；或外国出口商（`FOREIGN`），配额租流出国外。

`analyze_trade` `analyze_trade(⟨需求⟩, ⟨供给⟩, ⟨情景⟩)`
比较自给自足、自由贸易与政策三种情况。`TradeComparisonResult` 包含三个 `TradeOutcome`（`autarky`、`free_trade`、`policy`），以及政策相对于自由贸易的 `deadweight_loss`。

`domestic_price` 国内市场价格。
`quantity_demanded` 该价格下的国内需求量。
`quantity_supplied` 该价格下的国内供给量。
`imports` 进口量。
`exports` 出口量。
`direction` `TradeDirection`：`IMPORT`、`EXPORT` 或 `AUTARKY`。
`consumer_surplus` 消费者剩余。
`producer_surplus` 生产者剩余。
`government_revenue` 政府收入。
`quota_rent` 配额产生的租。
`national_quota_rent` 配额租中留在国内的部分。
`total_surplus` 国民剩余。
`gains_from_trade` 相对于自给自足的增加。
`is_policy_binding` 关税或配额是否改变价格。

```
from principle_viz import TradeScenario, analyze_trade

demand = line_from_inverse(12.0, -1.0)   # autarky price 7
scenario = TradeScenario(world_price=4, tariff=2)
result = analyze_trade(demand, supply, scenario)
print(result.free_trade.imports, result.policy.imports)
# 6.0 2.0
print(result.policy.government_revenue, result.deadweight_loss)
# 4.0 4.0
```

关税 2 时，国内价格为 6，政府收入为 $2 \times 2 = 4$。配额 2 单位时价格同为 6，配额租 4 归 `quota_rent_recipient` 指定的一方。

`MarketFigure.add_trade` `add_trade(⟨结果⟩)`
画出世界价格线并命名为 p_w，标出政策价格（$p_w + t$ 或 p_q），在数量轴上标 Q_s 与 Q_d 并于下方加上 “Imports” 或 “Exports” 括号，再以命名的矩形表示关税收入或配额租（参见图 16、图 17、图 18）。

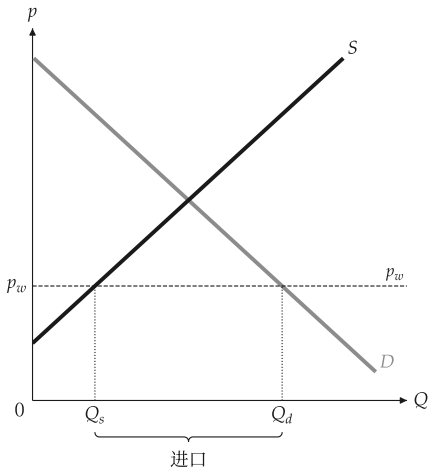


图 16
自由贸易下的进口

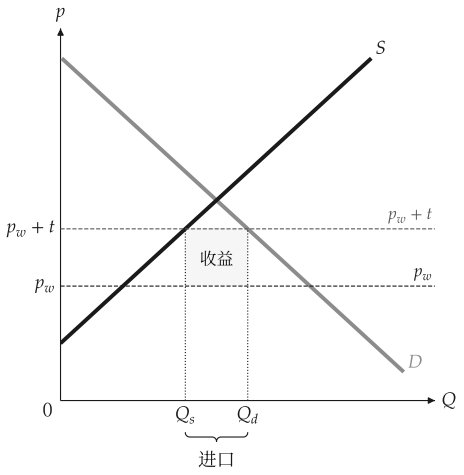


图 17
进口关税

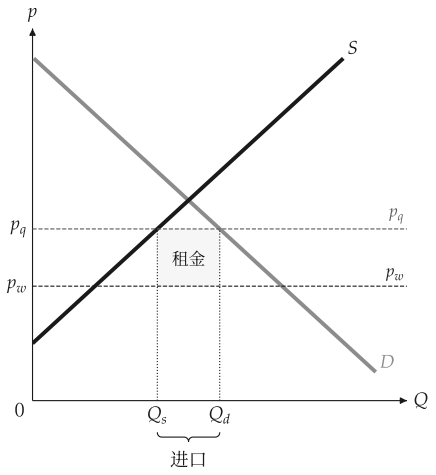


图 18
有约束的进口配额

第 12 节　市场失灵

12.1　外部性

生产对第三方造成成本，或消费为第三方带来利益时，市场数量会偏离社会最优数量。在最优数量处征收等于外部效应的庇古税或给予庇古补贴，可恢复最优数量（Pigou, 1920）。

`ExternalityScenario` `ExternalityScenario(marginal_external_cost=0.0, marginal_external_benefit=0.0)`

固定的边际外部成本（加到供给上得到边际社会成本），以及／或边际外部利益（加到需求上得到边际社会利益）。两者都不得为负。

`analyze_externality` `analyze_externality(⟨需求⟩, ⟨供给⟩, ⟨情景⟩)`

私人与社会的结果：`private_equilibrium`、`social_equilibrium`、`social_demand`、`social_supply`、`corrective_tax`、`corrective_subsidy`、`quantity_distortion` 与 `deadweight_loss`。

```
from principle_viz import ExternalityScenario, analyze_externality

demand = line_from_inverse(12.0, -1.0)
scenario = ExternalityScenario(marginal_external_cost=2)
result = analyze_externality(demand, supply, scenario)
print(result.private_equilibrium.q_star,
result.social_equilibrium.q_star)
# 5.0 4.0
print(result.corrective_tax, result.deadweight_loss)
# 2.0 1.0
```

`MarketFigure.add_externality` `add_externality(⟨结果⟩)`

画出社会曲线，在数量轴上标出 Q_m（市场）与 Q^*（最优），在 Q^* 处标出横跨差距的矫正税 t 或补贴 s，并为无谓损失填色（参见图 19、图 20）。

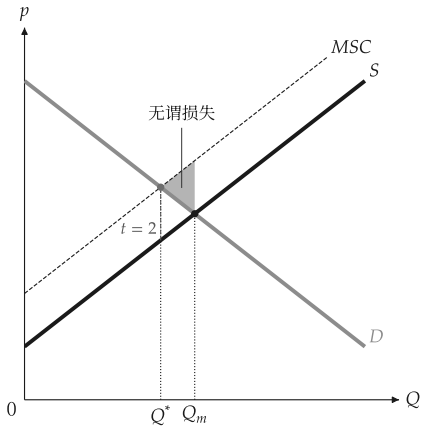


图 19
负外部性

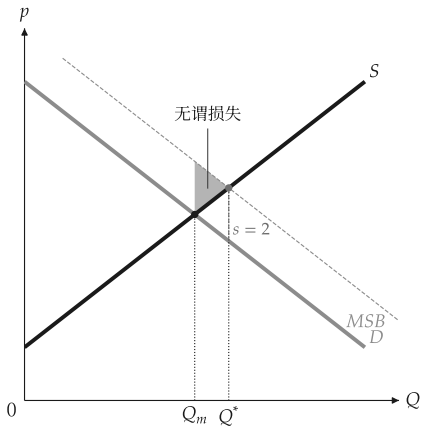


图 20
正外部性

12.2　公共资源

analyze_common_resource　`analyze_common_resource(⟨benefit⟩, ⟨cost⟩, *, marginal_congestion_cost)`
将拥挤或耗竭视为边际外部成本：开放获取时，资源会被使用到边际利益等于私人成本为止，超过有效率的水平 (Hardin, 1968)。结果包含 `open_access_equilibrium`、`efficient_equilibrium`、`social_cost`、`overuse`、`corrective_fee` 与 `deadweight_loss`。

以 $MB = 12 - Q$、$MPC = 2 + Q$、拥挤成本 3 为例，开放获取的使用量为 5 单位，有效率的水平为 3.5，`corrective_fee` 为 3。

MarketFigure.add_common_resource　`add_common_resource(⟨结果⟩)`

画出社会成本，标出 Q^* 与 Q_{open}，并为无谓损失填色。在 `add_curves()` 中将曲线命名为 *MB* 与 *MPC*。参见图 21。

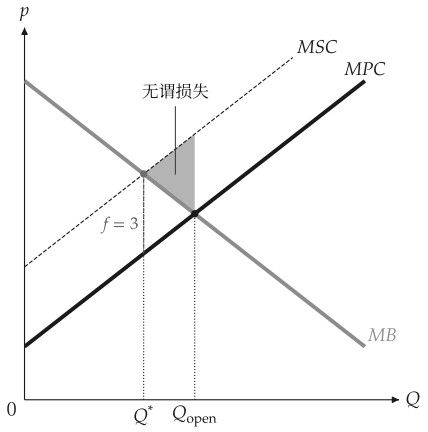


图 21
共有资源的过度使用

12.3　公共物品

IndividualBenefit　`IndividualBenefit(name, marginal_benefit)`

analyze_public_good　`analyze_public_good(⟨个人曲线⟩, ⟨cost⟩, *, samples=101)`

每个人都消费公共物品的全部数量，因此边际利益垂直加总。有效率的数量使边际利益之和等于边际成本 (Samuelson, 1954)；私人提供则停在最高的个人边际利益等于边际成本之处。结果包含 `efficient_quantity`、`efficient_marginal_value`、`private_provision_quantity`、`free_rider_gap` 与采样点 `points`。

```
from principle_viz import IndividualBenefit, analyze_public_good

result = analyze_public_good(
    (
        IndividualBenefit("$MB_A$", line_from_inverse(8, -1)),
        IndividualBenefit("$MB_B$", line_from_inverse(6, -1)),
    ),
    line_from_inverse(5, 0),  # constant marginal cost of 5
)
print(result.efficient_quantity, result.private_provision_quantity)
# 4.5 3.0
```

public_good_canvas　`public_good_canvas(⟨结果⟩, *, theme=None, labels=None, visibility=None)`

位于 `principle_viz.visuals.market_failures` 的 mosaickit 画布，画出各人的边际利益、其垂直加总、边际成本，并在数量轴上标出 Q_p 与 Q^*（参见图 22）。

第 13 节　劳动与可贷资金

13.1　最低工资

analyze_minimum_wage　`analyze_minimum_wage(⟨需求⟩, ⟨供给⟩, minimum_wage)`

设有工资下限的竞争劳动市场。`MinimumWageResult` 包含 `equilibrium`、`minimum_wage`、`is_binding`、`labor_demanded`、`labor_supplied`、`employment`（两者中较小者）、`unemployment`（两者之差）与 `wage_bill`。

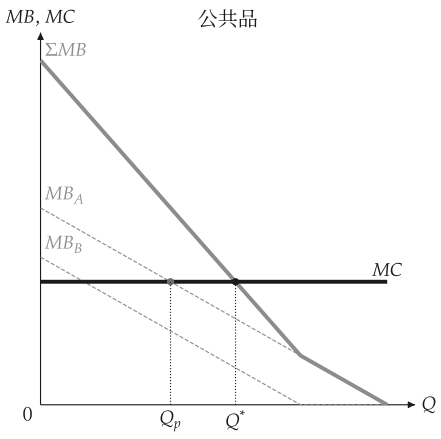


图 22
边际利益的垂直加总

```python
from principle_viz import analyze_minimum_wage

labor_demand = line_from_inverse(12, -1)
labor_supply = line_from_inverse(2, 1)
labor = analyze_minimum_wage(
    labor_demand, labor_supply, minimum_wage=9,
)
print(labor.employment, labor.unemployment)  # 3.0 4.0
```

`MarketFigure.add_minimum_wage` `add_minimum_wage(⟨结果⟩, *, gap_brace="line")`

以价格下限的方式绘制工资下限（详见第 10 节）：“Minimum wage” 线、工资轴上的 $w_{\min}$，有约束时另标出 L_d 与 L_s 及 “Unemployment” 括号。坐标轴命名为 L 与 w，曲线命名为 D_L 与 S_L（参见图 23）：

```python
fig = MarketFigure(x_max=11, y_max=14, x_label="L", y_label="w")
fig.add_curves(
    labor_demand, labor_supply, q_max=10,
    demand_label="$D_L$", supply_label="$S_L$",
)
fig.add_minimum_wage(labor)
```

`labor_demanded` 为 3，`labor_supplied` 为 7，`employment` 为 3，`unemployment` 为 4。

13.2　可贷资金

`LoanableFundsScenario` `LoanableFundsScenario(savings_quantity_shift=0.0, investment_quantity_shift=0.0, government_borrowing=0.0)`

储蓄（供给）与投资（需求）以数量衡量的水平移动。政府借款（不得为负）加入可贷资金的需求。

`analyze_loanable_funds` `analyze_loanable_funds(⟨savings⟩, ⟨investment⟩, ⟨情景⟩)`

移动前后的可贷资金市场：`baseline_equilibrium`、`shifted_equilibrium`、`shifted_savings`、`shifted_investment_demand`、`private_investment_after`、`interest_rate_change`，以及 `crowding_out`，即利率上升所挤出的民间投资。

```python
from principle_viz import (
    LoanableFundsScenario, analyze_loanable_funds,
```

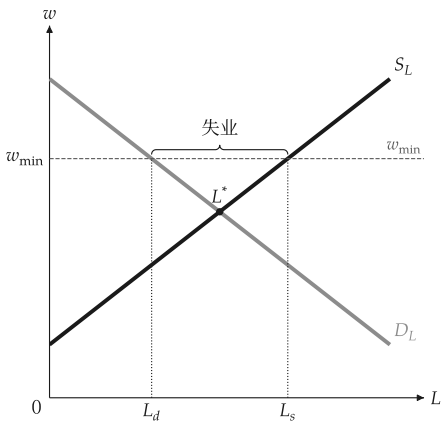


图 23
有约束的最低工资

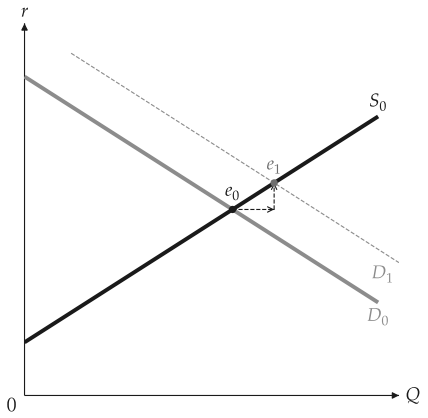


图 24
政府借款挤出投资

)

```
savings = line_from_inverse(2, 0.5)
investment = line_from_inverse(12, -0.5)
scenario = LoanableFundsScenario(government_borrowing=4)
funds = analyze_loanable_funds(savings, investment, scenario)
print(funds.interest_rate_change, funds.crowding_out)  # 1.0 2.0
```

`interest_rate_change` 为 1.0，`crowding_out` 为 2.0。

`MarketFigure.add_loanable_funds`　`add_loanable_funds(⟨结果⟩)`

画出移动后的曲线并命名为 D_1 或 S_1，以及两个均衡点与其间的移动。价格轴命名为 r（参见图 24）。

第 14 节　生产可能性曲线

`ProductionPossibilitiesFrontier`　`ProductionPossibilitiesFrontier(x_intercept, y_intercept, curvature=1.0, x_good="Good X", y_good="Good Y")`

通过截距 X 与 Y 的生产可能性曲线

$$y = Y(1 - (x/X)^c).$$

曲率 $c = 1$ 为直线（机会成本固定）；$c > 1$ 时曲线向外凸出（机会成本递增）。截距与曲率必须为正且 $c \geq 1$，否则抛出 `PPFError`。

`y_at(x)`　第一种商品生产 x 单位时 y 的产量。

`opportunity_cost_x(x)`　多生产一单位 x 所放弃的 y，即 $|\mathrm{d}y/\,\mathrm{d}x|$。

`assess(x, y)`　返回 `PointStatus`：曲线上为 `EFFICIENT`，曲线内为 `INEFFICIENT`，曲线外为 `UNATTAINABLE`。

`PointStatus`　`PointStatus.EFFICIENT, INEFFICIENT, UNATTAINABLE`

点相对于边界的位置。

`EFFICIENT`　在边界上。

PointStatus

`INEFFICIENT` 在边界内侧。

PointStatus

`UNATTAINABLE` 在边界外侧。

PointStatus

```python
from principle_viz import ProductionPossibilitiesFrontier

ppf = ProductionPossibilitiesFrontier(
    10, 8, curvature=2,
    x_good="Consumer goods", y_good="Capital goods",
)
print(ppf.y_at(6), ppf.opportunity_cost_x(6))  # 5.12 0.96
print(ppf.assess(4, 3), ppf.assess(7, 6))
# PointStatus.INEFFICIENT PointStatus.UNATTAINABLE
```

`analyze_ppf` `analyze_ppf(⟨生产可能性曲线⟩, points=(), *, samples=101)`

`ppf_canvas` `ppf_canvas(⟨结果⟩, *, theme=None, labels=None, visibility=None)`
在曲线上采样，并评估以 `(x, y, label)` 三元组给定的点；`principle_viz.visuals.ppf` 中的 `ppf_canvas()` 画出曲线、为可达成的区域填色并标出各点（参见图 25）。

```python
from principle_viz import analyze_ppf
from principle_viz.visuals.ppf import ppf_canvas

points = ((6, ppf.y_at(6), "A"), (4, 3, "B"), (7, 6, "C"))
result = analyze_ppf(ppf, points=points)
ppf_canvas(result).save("ppf_points.png")
```

`PPFGrowthScenario` `PPFGrowthScenario(x_growth_rate=0.0, y_growth_rate=0.0)`

`analyze_ppf_growth` `analyze_ppf_growth(⟨生产可能性曲线⟩, ⟨情景⟩, *, samples=101)`

`ppf_growth_canvas` `ppf_growth_canvas(⟨结果⟩, *, theme=None, labels=None, visibility=None)`
经济成长使各截距乘以一加上对应的成长率；结果包含 `baseline` 与 `shifted` 两条曲线及其采样点。`ppf_growth_canvas()` 画出两条曲线并命名为 PPF_0 与 PPF_1（参见图 26）。

14.1　比较优势

`compare_linear_ppfs` `compare_linear_ppfs(name_a, frontier_a, name_b, frontier_b)`
比较两条直线型生产可能性曲线：各生产者生产 x 的机会成本，以及各商品的比较优势归属（成本相同时为 `"tie"`）。曲线型的生产可能性曲线会抛出 `PPFError`。

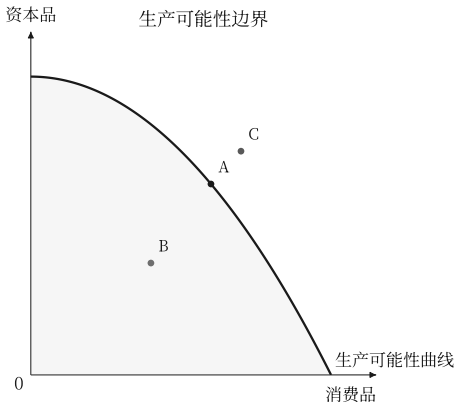


图 25
有效率与无效率的点

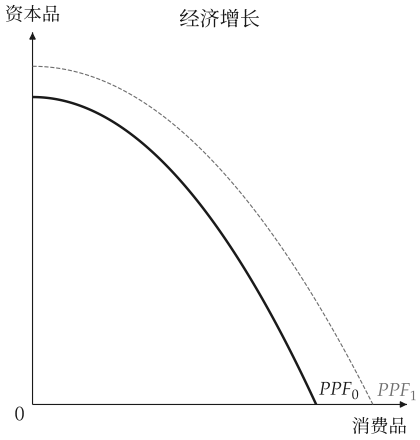


图 26
消费品与资本品增长

```
from principle_viz import compare_linear_ppfs

ann = ProductionPossibilitiesFrontier(10, 5)  # x costs 0.5 y
bob = ProductionPossibilitiesFrontier(6, 6)   # x costs 1 y
result = compare_linear_ppfs("Ann", ann, "Bob", bob)
print(result.comparative_advantage_x, result.comparative_advantage_y)
# Ann Bob
```

x 的比较优势属于 Ann（0.5 对 1），y 属于 Bob（1 对 2）。

第 15 节　图形

本手册的每张图都是 `MarketFigure`，或由 `ppf_canvas()` 等函数创建的 mosaickit 画布。两者的绘图区为正方形，坐标轴带箭头、无刻度与格线；曲线名称标在末端，面积名称写在区块内，数值标在坐标轴上，文字不遮住线、点、区块或其他文字。

15.1　MarketFigure

`MarketFigure`

```
from principle_viz import MarketFigure

fig = MarketFigure(
    x_max=20.0,
    y_max=20.0,
    x_label="Q",
    y_label="p",
    title="Market Diagram",
    theme=None,       # PlotTheme
    palette=None,     # "default", "nord", ...
    labels=None,      # {layer id: Label}
    visibility=None,  # {layer id: bool}
)
```

mosaickit 画布上的市场图，坐标轴由 0 延伸到 `x_max` 与 `y_max`。坐标轴名称除非是文字，否则以 LaTeX 数学排版；`title` 为空字符串时不画标题。

`theme`　颜色与线宽（详见第 15.3 节）。

`palette`　`PlotTheme.from_palette(palette)` 的简写；与 `theme` 同时给定时以 `theme` 为准。

`labels`　依图层 id 覆盖内建标签（详见第 15.2 节）。

`visibility`　依图层 id 显示或隐藏图层（详见第 15.2 节）。

MarketFigure.add_curves　`add_curves(⟨需求⟩, ⟨供给⟩, q_max, demand_label="$D$", supply_label="$S$")`

MarketFigure.add_equilibrium　`add_equilibrium(⟨结果⟩, label="$e^*$", color=None)`

由 $Q = 0$ 到 `q_max` 画出需求与供给并在末端命名；以实心点与标签标出均衡。

各主题另有专属方法，说明见对应章节（参见表 3）。

表 3　`MarketFigure` 的主题方法

方法	章节
`add_comparative_statics`	第 4.3 节
`add_discrete_curves`、`add_discrete_equilibrium`	第 5 节
`add_welfare`、`add_welfare_transition`	第 8 节
`add_tax_transform`、`add_tax_comparison`、`add_subsidy_comparison`	第 9 节
`add_price_control`	第 10 节
`add_trade`	第 11 节
`add_externality`、`add_common_resource`	第 12 节
`add_minimum_wage`、`add_loanable_funds`	第 13 节

MarketFigure.finalize　`finalize(legend=False)`

MarketFigure.save　`save(⟨路径⟩, dpi=150)`

`finalize()` 隐藏会把福利区块切成两半的辅助线（数值仍标在坐标轴上），只有在 `legend=True` 时才加入图例。`save()` 依扩展名写出 PNG、SVG 或 PDF，并创建不存在的目录。`close()` 不做任何事，仅为兼容而保留。

MarketFigure.add_layer　`add_layer(⟨layer⟩)`

MarketFigure.add_layers　`add_layers(⟨layers⟩)`

加入任意 mosaickit 图层，用于主题方法未提供的注记。同一数值的坐标轴标记会取代先前的标记。完成的场景可由 `fig.scene` 获得。

MarketFigure.add_metrics　`add_metrics(⟨数值⟩, *, title=None, location="upper right")`

以名称与数值组成的文字框，供笔记本与调试使用。教学用图不放此框，数值应写在正文中。

15.2　标签与图层

图中的每条线、每个点、每个区块与每段文字都是一个图层，各有固定的 id，例如 `market.demand`、`market.demand.label` 或 `market.welfare.dwl`。id 依图形结构命名：坐标轴为 `axes.*`，各元素为 `market.<部分>`，命名该元素的文字再加上 `.label` 后缀。`fig.layer_ids` 列出所有图层；`fig.label_ids` 列出可设置的文字（标签、坐标轴标记与括号）。

`Label` `Label(text=None, visible=None, offset=None, top=None, bottom=None, left=None, right=None)`

新增：*v0.10.1*　更新：*v0.12.1*

覆盖一个内建标签。值为 `None` 的字段保留软件包的默认值。指定 `offset`（单位为点）时，标签不再自动放置，改为精确移动该距离。`top`、`bottom`、`left` 与 `right` 则在标签定位之后再移动，位置仍由 principle-viz 决定，只做细部调整。

`MarketFigure.configure_label` `configure_label(⟨图层 *id*⟩, label=None, *, text=None, visible=None, offset=None, top=None, bottom=None, left=None, right=None)`

`MarketFigure.configure_layer` `configure_layer(⟨图层 *id*⟩, *, visible)`

新增：*v0.10.1*

改名、隐藏或移动一个标签（id 可省略结尾的 `.label`），或显示、隐藏任一图层。`hide(*ids)` 与 `show(*ids)` 一次切换多个图层。每个方法都返回图形本身。

同样的覆盖也可在创建图形时以 `labels=` 与 `visibility=` 对应表传入，之后加入的图层也会套用。加总图与各画布函数（`ppf_canvas()`、`public_good_canvas()` 等）接受同样的两个参数。

参见图 27：

```
from principle_viz import Label, MarketFigure

fig = MarketFigure(
    x_max=12, y_max=12,
    labels={
        "market.demand.label": Label(text="Demand"),
        "market.supply.label": Label(text="Supply"),
    },
)
fig.add_curves(demand, supply, q_max=10)
fig.add_equilibrium(eq)
fig.configure_label("market.equilibrium", text="$e^*$")
fig.hide("axes.origin.label")
fig.finalize()
```

微调标签

距离可为点数，或带单位的字符串：`"10pt"`、`"0.5em"`（以标签字号为单位）或 `"2mm"`。`top` 与 `bottom` 可以合并使用，`right` 与 `left` 亦同。微调在自动定位或 `offset` 之后应

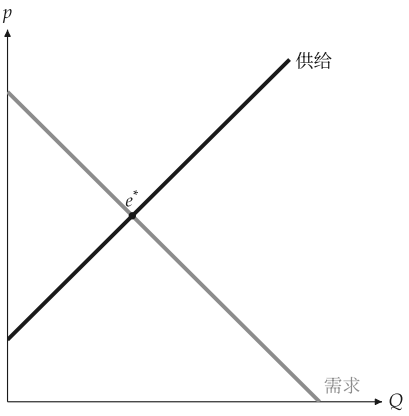


图 27
改名的曲线与均衡点

用；之后加入的标签仍会避开移动后的标签，未设置微调的图形保持不变。坐标轴刻度标签不支持微调。均衡点标签的 id 为 `market.equilibrium`：

```
from principle_viz import Label, MarketFigure

fig = MarketFigure(labels={"market.equilibrium": Label(top="4pt")})
fig.configure_label("market.supply", Label(right="8pt"))
fig.configure_label("market.demand", text="$D_0$", left="0.5em")
```

15.3　配色与主题

`ColorModel` 为每个经济角色指定颜色；`PlotTheme` 再加上线宽与坐标轴选项，并将两者编译为 mosaickit 主题。内置配色就是所有 EconViz 软件包共享的主题（表 4），同一名称在 utility-viz 与 principle-viz 中外观一致。

`ColorModel`　`ColorModel(name, axis_color, label_color, demand_color, supply_color, ...)` 具名的颜色角色：`axis_color`、`label_color`、`demand_color`、`supply_color`、`baseline_color`、`shifted_color`、`tax_color`、`control_color`、`cs_color`、`ps_color`、`tax_revenue_color`、`dwl_color` 与 `arrow_color`。每个值可为 `"#hex"` 或 `"blue"` 等 mosaickit 调色板名称。

表 4
内置配色

名称	特色与用途
`manual`	EconViz 手册的灰阶：黑色坐标轴、供给曲线与箭头，灰色需求曲线与剩余区域，适合打印与讲义
`default`	需求蓝、供给绿、无谓损失红，税收与价格管制橙；剩余区域沿用曲线颜色，适合日常使用
`colorblind`	与 `default` 相同的配色，以明确名称标示
`monochrome`	黑色曲线搭配灰阶区域，适合黑白打印
`nord`	采用 Nord 色盘的冷色系，适合屏幕与幻灯片
`paper`	柔和颜色，适合论文
`presentation`	颜色更鲜明，线条放大为 1.5 倍、均衡点放大为 1.3 倍，适合投影

共享主题为各个角色指定颜色，`principle_viz.visuals.roles` 则决定各元素使用的角色：坐标轴、标签、基准线与箭头使用 ink，需求使用 primary，供给使用 secondary，移动后的曲线使用 tertiary，税收与价格管制使用 intervention，消费者剩余、生产者剩余、无谓损失与税收收入分别使用 light、mid、dark 与 faint 填色。主题的线条与标记比例会乘以 principle-viz 自身的线宽与均衡点大小。内置主题不含深色主题。

`list_color_models()` 返回所有名称，`get_color_model(name)` 返回对应的配色；另导出 `DEFAULT_COLOR_MODEL`、`COLORBLIND_COLOR_MODEL`、`NORD_COLOR_MODEL` 与 `MONOCHROME_COLOR_MODEL`。

PlotTheme　PlotTheme(color_model=..., demand_linewidth=3.5, supply_linewidth=3.5, ...)

PlotTheme.from_palette(〈名称〉, **options)

配色加上线宽（demand_linewidth、supply_linewidth、shifted_linewidth、tax_linewidth、arrow_linewidth、dashed_linewidth）、equilibrium_marker_size，以及开关 show_grid、show_ticks、show_axis_arrows 与 show_origin_label。from_palette() 以内建配色为起点，并可设置上述任一字段。

字体有三个字段。font 是 "sans"（默认）、"serif" 或 "mono"，已安装的字体名称（如 "Arial"）会报错。fonts 是 mosaickit.TextFonts，用字体文件取代三者之一，没给的沿用 mosaickit 的 DejaVu 默认。math_font 是 MathFont 或 OpenType 数学字体的路径，经由 glyphkit 排版 $...$。font_roles 把样式角色对应到通用字体，例如 {"axes": "mono"}。系统字体不会被搜索，请直接给路径。

```
from mosaickit import TextFonts
from principle_viz import MarketFigure, PlotTheme

theme = PlotTheme(
    font="serif",
    fonts=TextFonts(serif=["fonts/Pagella-Regular.otf", "fonts/Pagella-
Bold.otf"]),
    math_font="fonts/Pagella-Math.otf",
)
fig = MarketFigure(x_max=12, y_max=12, theme=theme)
```

以 palette 参数选用配色（参见图 28）：

```
fig = MarketFigure(x_max=12, y_max=12, palette="monochrome")
```

15.4　配置文件

principle-viz 读取所有 EconViz 软件包共享的配置文件 econ-viz.toml，并由 mosaickit 读取与检查。顶层放置所有软件包共享的配置：起始主题（base，可选表 4 中的名称）、字体，以及坐标轴与标签颜色。principle-viz 自身的配置放在 [principle] 表中；其他软件包的表（例如 [utility]）会被跳过。

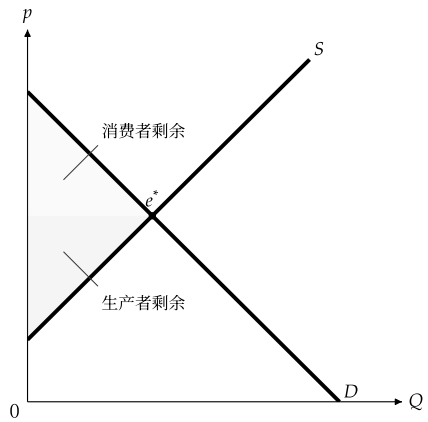


图 28
monochrome 配色

```toml
base = "manual"       # 共享主题名称

[font]
text = "serif"
math = "fonts/texgyrepagella-math.otf"

[fonts]
serif = ["fonts/NotoSerifSC-Regular.ttf", "fonts/NotoSerifSC-Bold.ttf"]

[principle]
language = "zh-CN"    # 内置图形文字

[principle.color]     # 配色角色名称
demand = "#8c8c8c"

[principle.stroke]    # 线宽以点为单位
demand = 3.5

[principle.marker]
equilibrium = 6.5     # 均衡点直径
```

`base = "manual"` 即为本手册的黑白样式，无须逐一写出颜色与线宽。`[principle] palette` 仍可只为 principle-viz 设置配色，并优先于 `base`；`[principle.color]` 则设置单个元素。相对路径以配置文件所在文件夹为准。

`Config.load`
`Config.use`
`default_theme`

```python
from principle_viz import Config

Config.load("econ-viz.toml").use()
```

新增：*v0.12.0*

`Config.load()` 读取配置文件，`use()` 将其设为默认值：未传入 `theme=` 或 `palette=` 的图形都会采用，`default_theme()` 则返回这些图形使用的主题。直接传入图形的 `theme=` 或 `palette=` 仍优先。`mosaickit.econ_toml.use("econ-viz.toml")` 会一次为所有已安装的 EconViz 软件包加载此文件。

配置文件写错时会抛出 `mosaickit.ConfigurationError`，指出文件与表，并在有相近名称时提示可能的正确写法：

```
econ-viz.toml: [principle.color] unknown key 'demnad'; did you mean
'demand'?
```

15.5　画布

`Canvas`

mosaickit 的 `Canvas`，并具备与 `MarketFigure` 相同的 `labels=`、`visibility=`、`configure_label()`、`configure_layer()`、`hide()`、`show()`、`layer_ids` 与 `label_ids`。生产可能性曲线、公共物品与总收益各章的画布函数都返回此类别；以 `save()` 保存，或以 `mosaickit.CanvasGrid` 组合多张画布。

第 16 节　命令行界面

`principle-viz` 命令在命令行运行本手册中的计算，并以 JSON 打印结果；`--output` 会另外写成文件。此命令不绘图。可将其安装为工具（详见第 2.4 节），或在项目中于命令前加上 `uv run`。

```
principle-viz equilibrium \
  --demand-intercept 10 --demand-slope -1 \
  --supply-intercept 2 --supply-slope 1
```

```
{
  "q_star": 4.0,
  "p_star": 6.0,
  "is_valid_market": true,
  "notes": []
}
```

16.1　市场参数

多数命令以反函数形式 $p = a + bQ$ 的两条直线描述市场：

`--demand-intercept` 需求的价格截距 a。

`--demand-slope` 需求的斜率 b。

`--supply-intercept` 供给的价格截距 a。

`--supply-slope` 供给的斜率 b。

`--output` 另将 JSON 写入此文件，并创建所需目录。

16.2　命令

各命令对应的计算与选项如表 5；每个命令的计算说明见对应章节。

表 5　命令行命令

命令	计算与其他选项	章节
`equilibrium`	线性均衡	第 4 节
`shift`	比较静态：`--demand-delta-intercept`、`--demand-delta-slope`、`--supply-delta-intercept`、`--supply-delta-slope`	第 4.3 节
`discrete`	离散市场：`--demand-values`、`--supply-values`、`--price-rule {midpoint,lower,upper}`	第 5 节
`elasticity`	点弹性：`--intercept`、`--slope`、`--quantity`；加上 `--q1`、`--p1` 计算弧弹性	第 7 节
`revenue`	弹性与总收益表：`--samples`	第 7 节
`welfare`	剩余：`--policy {baseline,tax,subsidy,control}` 与该政策的选项	第 8 节
`report-dwl`	单列无谓损失报表：`--policy {tax,subsidy,control}`、`--csv`	第 8 节
`tax`	`--tax-type {fixed,per_unit,ad_valorem}`、`--amount`、`--tax-on {consumer,producer}`	第 9 节
`subsidy`	`--amount`、`--subsidy-to {consumer,producer}`	第 9 节
`controls`	`--control-type {ceiling,floor}`、`--control-price`	第 10 节

命令	计算与其他选项	章节
`trade`	`--world-price`、`--tariff`、`--import-quota`、`--quota-rent-recipient`	第 11 节
`externality`	`--external-cost`、`--external-benefit`	第 12 节
`common-resource`	`--congestion-cost`	第 12 节
`public-good`	`--benefit-intercepts`、`--benefit-slopes`、`--cost-intercept`、`--cost-slope`、`--samples`	第 12 节
`minimum-wage`	`--labor-demand-*`、`--labor-supply-*`、`--minimum-wage`	第 13 节
`loanable-funds`	`--savings-*`、`--investment-*`、`--savings-shift`、`--investment-shift`、`--government-borrowing`	第 13 节
`ppf`	`--x-intercept`、`--y-intercept`、`--curvature`、`--x-good`、`--y-good`、`--x-growth`、`--y-growth`、`--samples`	第 14 节

`principle-viz <command> --help` 列出命令的所有选项。

```
principle-viz tax \
  --demand-intercept 10 --demand-slope -1 \
  --supply-intercept 2 --supply-slope 1 \
  --tax-type per_unit --amount 1 --tax-on producer
```

输出包含未征税的均衡（`baseline_equilibrium`）、征税后的均衡（`post_tax`）与各项变动，与 `compare_tax_scenario()` 的返回值相同：本例数量为 3.5，消费者价格 6.5，生产者价格 5.5。

更新纪录

本章列出影响软件包使用的版本变更，包含以 `principle-econ` 名称发布的版本，不含仅修改文档的版本；各条目依 l3doc 惯例标注于它所描述的功能旁，页码即该功能的实际页码。

v0.12.1 (2026-10-23)

`Label`：添加 `top`、`bottom`、`left` 与 `right`，可在标签定位后微调单个标签 ····· 33

`ColorModel`：配色改为 EconViz 共享主题，同一名称在各 EconViz 软件包中外观一致；添加 `paper` 与 `presentation` ····· 35

`Config`：改由 mosaickit 按 principle-viz 声明的结构读取并检查文件；共享的 `base` 决定起始主题 ····· 36

v0.12.0 (2026-10-23)

`Config`：`Config.load("econ-viz.toml").use()` 读取所有 EconViz 软件包共享的配置文件 ····· 36

v0.11.2 (2026-10-22)

`ColorModel`：添加 `manual` 配色，即本手册的灰阶 ····· 35

v0.11.0 (2026-10-21)

`PlotTheme`：新增 `font`、`fonts`、`math_font` 与 `font_roles` ····· 36

v0.10.1 (2026-10-03)

一般：CI 测试 Python 3.10–3.13 ····· 2

`demand_aggregation_figure`：市场曲线在折点处标出当时有效的需求 ····· 12

`Label`：每个内建标签都可改名、隐藏或移动：新增 `Label`、`labels=`、`configure_label()` 与 `label_ids` ····· 33

`MarketFigure.configure_layer`：每个图层都可隐藏或显示：新增 `visibility=`、`configure_layer()`、`hide()`、`show()` 与 `layer_ids` ····· 33

v0.10.0 (2026-10-03)

一般：由 `principle-econ` 更名为 principle-viz，加入 EconViz 系列；Python 软件包名称改为 `principle_viz` ····· 2

图形改以 mosaickit 绘制（`mosaickit>=0.5.1,<0.6.0`） ····· 2

项目工具由 Poetry 改为 uv ····· 2

`principle-viz`：唯一的运行时依赖为 mosaickit ····· 3

`add_comparative_statics`：只重画移动的曲线；移动后的曲线命名为 D_1 / S_1（`demand_label` / `supply_label`） ····· 7

移动箭头改为黑色细虚线 ····· 7

`MarketFigure.add_discrete_curves`：可只传入需求表、只传入供给表，或两者都传入 ····· 9

空心端点改为不透明的白色填色，并以细虚线连到下一阶 ····· 9

`market_demand`：个人线性曲线水平加总为市场需求与供给，并新增 `PiecewiseLinear`、`solve_piecewise_equilibrium()` 与 `piecewise_surplus()` ····· 11

`DiscreteDemand.combine`：离散表新增 `combine()` 与 `quantity_at()` ····· 11

`demand_aggregation_figure`：需求与供给（线性与离散）的**个人 | 个人 | 市场**图，返回 `AggregationFigure` ····· 12

`link_price=True` 让价格线横跨所有面板 ···· 12

`MarketFigure.add_welfare`：以文字命名各区块（“Consumer surplus”、“CS” 等），写在区块内、改用缩写或以引线标示；`labels=False` 只填色 ····· 15

消费者剩余与生产者剩余沿用需求与供给的色相；税收只标示文字、不填色 ····· 15

`MarketFigure.add_tax_transform`：征税后的曲线命名为 $S + t$ / $D - t$；旋转标签移到箭头尾端 ····· 17

`MarketFigure.add_tax_comparison`：在价格轴上标出 p_d、p_0 与 p_s，并加上 “Tax” 括号；新增 `brace_side` 与 `notes` ····· 19

`MarketFigure.add_subsidy_comparison`：补贴楔子加上 “Subsidy” 括号，并标示补贴成本 · 20

`MarketFigure.add_price_control`：有效的管制直接命名管制线，标出 p_c、Q_d 与 Q_s，并在管制线上以括号标示 “Shortage” 或 “Surplus”；`gap_brace="axis"` 改在数量轴下方标示 ····· 21

`MarketFigure.add_trade`：标出 Q_s / Q_d，并在数量轴下方以括号标示 “Imports” / “Exports”；政策价格标为 $p_w + t$ 或 p_q ····· 24

`MarketFigure.add_externality`：社会曲线命名为 *MSC* / *MSB*；Q_m 与 Q^* 标在数量轴上；矫正税与补贴改为横跨 Q^* 处的差距 ····· 26

`MarketFigure.add_minimum_wage`：以价格下限的方式绘制：“Minimum wage” 线、$w_{\min}$、L_d / L_s 与 “Unemployment” 括号 ····· 28

`MarketFigure`：图形改为正方形（150 dpi 下 6 × 6 英吋），背景为不透明白色；直线曲线止于价格轴的 92% ····· 32

曲线名称标在可见端点旁，不再放进图例 ···· 32

坐标轴名称默认为 p 与 Q，以斜体数学字体放在箭头外侧；标题不再使用粗体 ····· 32

均衡与政策标签由 mosaickit 放在点旁；拥挤处的数量改标在数量轴上 ····· 32

`MarketFigure.finalize`：默认不加图例；`finalize(legend=True)` 才会加入 ····· 32

`ColorModel`：`default` 配色改用 mosaickit 的色相：需求蓝、供给红、无谓损失青 ····· 35

`PlotTheme`：供需曲线宽 3.5 pt、坐标轴 1.0 pt、均衡点直径 6.5 pt ····· 35

v0.1.0 (2026-04-25)

一般：以 `principle-econ` 名称首次发布：线性市场、税收、福利、绘图与命令行界面 ····· 2

`TaxScenario`：`fixed`、`per_unit` 与 `ad_valorem` 三种税，法定纳税人可为买方或卖方 ·· 17

`ColorModel`：`default`、`colorblind`、`nord` 与 `monochrome` 四种配色 ····· 35

`principle-viz`：命令行界面，输出 JSON ··· 37

命令索引

A

analyze_common_resource 27
analyze_externality 26
analyze_loanable_funds 29
analyze_minimum_wage 28
analyze_ppf 31
analyze_ppf_growth 31
analyze_public_good 28
analyze_trade 24

B

build_dwl_report 17
build_tax_visual_guide 18

C

Canvas 37
ColorModel 35
comparative_statics 7
compare_linear_ppfs 31
compare_subsidy_scenario 20
compare_surplus 16
compare_tax_scenario 18
compute_arc_elasticity 14
compute_discrete_surplus 10
compute_point_elasticity 14
compute_surplus 16
Config.load 37
Config.use 37

D

default_theme 37
demand_aggregation_figure 12
discrete_demand_aggregation_figure 12
discrete_supply_aggregation_figure 13
DiscreteDemand 9
DiscreteSupply 9

E

elasticity_revenue_canvases 15
elasticity_revenue_schedule 14
evaluate_price_control 22
ExternalityScenario 26

I

IndividualBenefit 28

L

Label 34
Line 6
line_from_inverse 5
LoanableFundsScenario 29

M

market_demand 11
market_supply 11
MarketFigure 5, 32
MarketFigure.add_* 5
MarketFigure.add_common_resource 27
MarketFigure.add_comparative_statics 8
MarketFigure.add_curves 33
MarketFigure.add_discrete_curves 10
MarketFigure.add_discrete_equilibrium 10
MarketFigure.add_equilibrium 33
MarketFigure.add_externality 26
MarketFigure.add_layer 33
MarketFigure.add_layers 33
MarketFigure.add_loanable_funds 30
MarketFigure.add_metrics 33
MarketFigure.add_minimum_wage 29
MarketFigure.add_price_control 22
MarketFigure.add_subsidy_comparison 21
MarketFigure.add_tax_comparison 19
MarketFigure.add_tax_transform 19
MarketFigure.add_trade 24
MarketFigure.add_welfare 16
MarketFigure.add_welfare_transition 17
MarketFigure.configure_label 34

MarketFigure.configure_layer 34
MarketFigure.finalize 5, 33
MarketFigure.save 5, 33
MarketOutcome 15

P

piecewise_surplus 12
PiecewiseLinear 11
PlotTheme 36
PointStatus 30
ppf_canvas 31
ppf_growth_canvas 31
PPFGrowthScenario 31
PriceControlScenario 21
PriceControlType 21
PrincipleVizError 6
ProductionPossibilitiesFrontier 30
public_good_canvas 28

S

ShiftScenario 7
ShiftSpec 7
solve_discrete_equilibrium 9
solve_equilibrium 5, 7
solve_piecewise_equilibrium 12
solve_subsidy_equilibrium 20
solve_tax_equilibrium 18
SubsidyScenario 20
SubsidyTo 20
supply_aggregation_figure 12

T

TaxOn 18
TaxScenario 17
TaxType 18
TradeScenario 24

参考文献

Hardin, G. (1968). The Tragedy of the Commons. *Science*, *162*(3859), 1243–1248. https://doi.org/10.1126/science.162.3859.1243

Mankiw, N. G. (2021). *Principles of Economics* (9th ed.). Cengage Learning.

Marshall, A. (1890). *Principles of Economics*. Macmillan.

Pigou, A. C. (1920). *The Economics of Welfare*. Macmillan.

Samuelson, P. A. (1954). The Pure Theory of Public Expenditure. *The Review of Economics and Statistics*, *36*(4), 387–389. https://doi.org/10.2307/1925895